

Basic Information

Student name: Yaotian Liu

Project title: RTLRepair-Agent: Verification-Guided Agentic RTL Generation

Repository / notebook link: <https://github.com/ytliu74/rtlrepair-agent>

Configuration location: README.md, .env.example, and examples/counter/.

Section 1. Problem Definition

For an RTL designer, convert a natural-language hardware specification and fixed module interface into synthesizable SystemVerilog. An external testbench evaluates the output. Success requires compilation and simulation exit codes of zero, an explicit `TEST_PASS`, and no `TEST_FAIL`. Syntax/interface errors, timeouts, functional mismatches, and missing pass markers are failures. Research question: can an LLM using compiler/simulator feedback achieve a higher functional pass rate than one-shot generation?

Section 2. Motivation and Project Scope

Generated RTL can contain syntax, timing, and functional errors. Compiler diagnostics and simulation mismatches supply objective feedback for an agentic generate–observe–repair workflow. Open-source Icarus Verilog makes evaluation feasible within a semester. Implemented now: one-shot generation and automated verification. Planned: bounded repair and comparison on additional RTL tasks. Synthesis/PPA feedback is a stretch goal; physical design and SoC-scale generation are outside the initial scope.

Section 3. Runnable Baseline

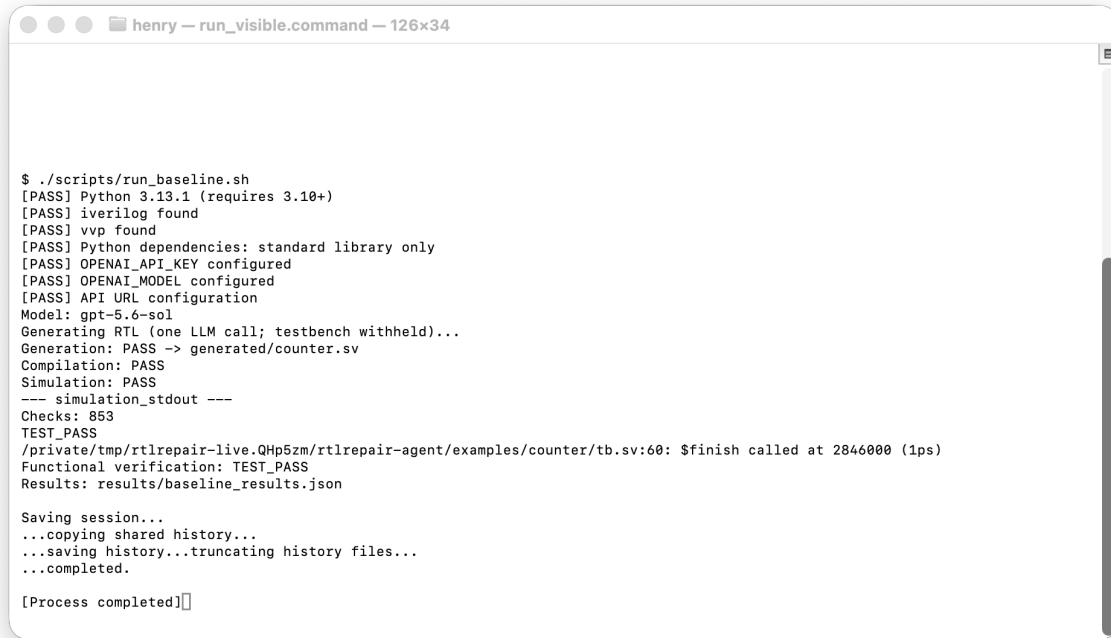
Python 3.10+ makes one OpenAI Chat Completions call to `gpt-5.6-sol`, cleans and saves the module, compiles with `iverilog -g2012`, and simulates with `vvp`. The model receives only the specification/interface; the testbench is withheld. `src/llm_client.py`, `rtl_generator.py`, `simulator.py`, and `baseline.py` implement the pipeline. Run `./scripts/run_baseline.sh`. This one-shot baseline is the experimental control because it cannot incorporate feedback or repair failures.

Section 4. Test Case and Baseline Output

Input: an 8-bit counter with `clk`, `reset`, `enable`, and `count`; rising-edge increments, synchronous reset to zero with priority, hold when disabled, and 255-to-0 wrap. The actual generated module uses `always_ff @(posedge clk)` with reset and enabled-increment branches. Two live calls on September 6, 2026 each compiled and simulated successfully: **853 checks; TEST_PASS**. Generation took 2.645 s initially and 1.942 s in a fresh public clone; each call reported 322 total tokens. These demonstrate reproducibility on one task, not general accuracy. `generated/counter.sv`, `results/baseline_results.json`, and `artifacts/baseline_output.txt` preserve the first run; `reproduction_* results/logs` and `generated/counter_reproduction.sv` preserve the second.

Section 5. Reproducibility and Run Instructions

Clone the repository above and enter `rtlrepair-agent`. Install Python 3.10+ and Icarus Verilog (`brew install icarus-verilog` on macOS). Run `python3 -m venv .venv`, `source .venv/bin/activate`, and `python -m pip install -r requirements.txt` (standard library only). Copy `.env.example` to `.env`; fill `OPENAI_API_KEY` and retain `OPENAI_MODEL=gpt-5.6-sol`. Leave optional `OPENAI_BASE_URL` blank for `https://api.openai.com/v1`. Run `./scripts/run_baseline.sh`. Inputs are `examples/counter/spec.txt` and `examples/counter/tb.sv`; outputs are listed above. API access and model availability are required. README includes Linux setup and offline testing; 17 tests pass.

A terminal window titled "henry — run_visible.command — 126x34" showing the output of a script. The script runs successfully, passing all checks and generating RTL. The output includes status messages for Python, IVerilog, and VVP, configuration for OpenAI API and model, and a final summary of the simulation and verification results.

```
$ ./scripts/run_baseline.sh
[PASS] Python 3.13.1 (requires 3.10+)
[PASS] iverilog found
[PASS] vvp found
[PASS] Python dependencies: standard library only
[PASS] OPENAI_API_KEY configured
[PASS] OPENAI_MODEL configured
[PASS] API URL configuration
Model: gpt-5.6-sol
Generating RTL (one LLM call; testbench withheld)...
Generation: PASS -> generated/counter.sv
Compilation: PASS
Simulation: PASS
--- simulation_stdout ---
Checks: 853
TEST_PASS
/private/tmp/rtlrepair-live.QHp5zm/rtlrepair-agent/examples/counter/tb.sv:60: $finish called at 2846000 (1ps)
Functional verification: TEST_PASS
Results: results/baseline_results.json

Saving session...
...copying shared history...
...saving history...truncating history files...
...completed.

[Process completed]
```

Figure 1: Actual Terminal capture of the fresh-clone run; full-resolution image is in artifacts/baseline_screenshot.png.

Section 6. Initial Evaluation Plan

Compare one-shot generation with bounded verification-guided repair using the same model, initial prompt, tasks, and independent testbenches across repeated trials. Primary metric: functional pass rate. Also measure compilation rate, iterations, calls, tokens, latency, and measurable cost. Include budget-matched independent resampling to distinguish feedback benefit from extra calls. Start with curated modules; consider VerilogEval and RTLLM later. Retain all failures.

Section 7. Limitations and Next Steps

One counter cannot establish generality. Simulation checks only covered behavior and does not prove synthesizability. Model nondeterminism, API availability, and testbench coverage limit repeatability and correctness. The baseline cannot repair errors. Next: implement failure diagnosis and bounded repair, expand the task set, and evaluate against the control; optionally add synthesis/PPA feedback.